

GRAPH NEURAL NETWORKS FOR SYSTEM CALL GRAPH-BASED INTRUSION DETECTION

Raphaël Faure & Tristan Beruard

CentraleSupélec

{raphael.faure2, tristan.beruard}@student-cs.fr

ABSTRACT

Intrusion detection at the host level is a critical problem in cybersecurity, especially with the increasing sophistication of attack. System call traces provide a fine-grained view of program execution and are widely used for anomaly detection. While most existing approaches model system calls as sequences, several works suggest that graph-based representations can better capture complex dependencies between system calls (Various (2018)). In this project, we propose to model system call traces as directed weighted graphs and evaluate different graph mining and representation learning approaches for intrusion detection. Given system call datasets such as ADFA-LD (Creech & Hu (2013)) and LID-DS et al. (2021), we aim to compare classical graph-based methods (e.g., PageRank, graph kernels, node embeddings) with modern Graph Neural Networks (GCN (Kipf & Welling (2017)), GraphSAGE, GAT) which have so far been only marginally explored in the context of system call-based intrusion detection. We will also investigate whether incorporating temporal information via dynamic graphs improves performance and if time allows it explore self-supervised learning strategies to address class imbalance and limited labeled data.

1 MOTIVATION AND PROBLEM DEFINITION

Intrusion detection systems (IDS) aim to identify malicious activity on a system. Host-based IDS rely on monitoring system-level events such as system calls, which are low-level interactions between programs and the operating system. These sequences provide a detailed and representation of program behavior.

Formally, we consider a dataset of system call traces corresponding to program executions, labeled as either normal or malicious. The objective is to learn a function:

$$f : \mathcal{G} \rightarrow \{0, 1\}$$

that maps a graph representation of a system call trace to a binary label (normal vs intrusion).

Traditional approaches model system calls as sequences and apply n-gram models, HMMs, or more recently, deep learning models such as LSTMs and Transformers. However, these approaches may fail to capture long-range dependencies and structural relationships between system calls.

In this project, we propose to model system call traces as graphs, where nodes represent system calls and edges represent transitions between calls. This formulation naturally captures interactions and recurring patterns in execution flows.

Given system call datasets (e.g., ADFA-LD, LID-DS), **we use** graph-based representations and graph learning methods, **to achieve** accurate and robust intrusion detection.

2 METHODOLOGY

Dataset and Graph Construction A system call occurs when a program requests a service from the operating system (e.g., reading a file, creating a process). Each execution trace is a sequence of such calls.

We construct a directed weighted graph for each execution:

- Nodes: unique system calls
- Edges: transition from call A to B
- Edge weights: frequency of transitions

Alternative constructions will also be explored: inclusion of timestamps (temporal graphs), inclusion of syscall arguments or return values.

We will use publicly available datasets:

- ADFA-LD (modern Linux dataset) Creech & Hu (2013)
- LID-DS (realistic and large-scale dataset) et al. (2021)
- UNM / DARPA (for comparison with older benchmarks)

Algorithms Given a graph representation $G = (V, E)$ of a system call trace, where nodes correspond to system calls and edges to transitions, the objective is to classify each execution as normal or malicious.

Classical Graph-based Approaches We first consider baseline methods based on handcrafted graph features. From each graph, we extract structural properties such as degree statistics, transition frequencies, and centrality measures.

In particular, PageRank can be used to identify important system calls and detect abnormal execution patterns through their distribution pag (2013). These features are then used with standard classifiers such as logistic regression or SVM.

Graph Representation Learning To avoid manual feature design, we consider representation learning methods. DeepWalk Perozzi et al. (2014) learns node embeddings via random walks, capturing local structural similarities between system calls. Graph2Vec Narayanan et al. (2017) extends this idea to whole graphs, producing fixed-size representations for each execution trace. These embeddings are then used as input to downstream classifiers.

Graph Neural Networks Graph Neural Networks (GNNs) learn node representations by iteratively aggregating information from neighbors. This allows the model to capture higher-order dependencies between system calls, which is crucial for identifying complex malicious behaviors.

We will evaluate architectures such as GCN Kipf & Welling (2017), GraphSAGE, and GAT. Node representations are aggregated into a graph-level embedding using a readout function, followed by a classifier..

Temporal Graph Modeling Static graphs ignore the temporal order of system calls, which is often critical. To address this, we consider dynamic graph models where interactions are timestamped.

We use a Temporal Graph Network (TGN) et al. (2020), where node representations evolve over time:

$$\mathbf{h}_v(t) = \text{GRU}(\mathbf{h}_v(t^-), \mathbf{m}_v(t))$$

where $\mathbf{m}_v(t)$ encodes the current interaction. This allows the model to capture both structural and temporal dependencies.

Learning Paradigms We consider three complementary learning settings.

Supervised learning. Models are trained directly on labeled graphs using standard classification losses.

Contrastive learning. To leverage unlabeled data, we use a contrastive objective that brings together representations of augmented versions of the same graph:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{z}, \mathbf{z}_j)/\tau)}$$

Anomaly detection. Finally, we explore settings where the model is trained mainly on normal data and detects deviations, which reflects real-world intrusion scenarios.

Prior Work Previous work has mainly focused on sequential modeling of system calls using statistical or neural sequence models. However, recent work suggests that graph-based representations better capture structural dependencies Various (2018).

Graph-based intrusion detection has been explored using handcrafted features or traditional methods such as PageRank pag (2013), but less work compares them systematically with modern GNNs. Additionally, temporal graph modeling remains underexplored in this domain.

Our work aims to fill this gap by providing a unified experimental comparison across multiple graph-based approaches.

3 EVALUATION

We will evaluate models using standard classification metrics: Accuracy, F1-score, ROC-AUC It could be interesting to also investigate:

- **Robustness:** sensitivity to perturbations in system call sequences (after a slight modification of the sequences, do the models still work?)
- **Generalization:** cross-dataset performance (train on ADFA-LD, test on LID-DS)
- **Online detection:** Implement incremental updates with streaming system calls

4 REFERENCES

REFERENCES

- An anomaly intrusion detection method based on pagerank algorithm. *IEEE*, 2013.
- Gideon Creech and Jiankun Hu. Adfa-ld: A new dataset for host-based intrusion detection. 2013.
- Kiss et al. A modern and sophisticated host-based intrusion detection dataset. 2021.
- Rossi et al. Temporal graph networks for deep learning on dynamic graphs. 2020.
- Thomas Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- Annamalai Narayanan, Muthu Chandramohan, et al. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005*, 2017.
- Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *KDD*, 2014.
- Various. Intrusion detection on system call graphs. 2018.